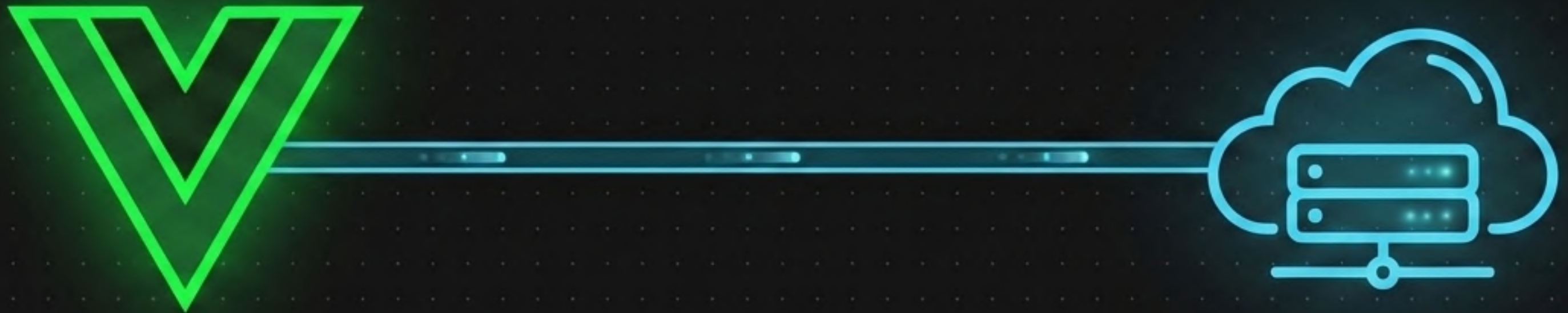


Consumo de APIs en Vue 3 con Axios

De la petición HTTP al renderizado de datos



El viaje de los datos

¿Cómo llega la información desde un servidor hasta la pantalla del usuario?



Axios vs Fetch

¿Por qué usamos una librería externa?

Característica	fetch() nativo	Axios
Transformación JSON	Manual (response.json()) ⚠️	Automática ⚡
Manejo de Errores	Requiere validar !response.ok ⚠️	Lanza error y ejecuta catch automáticamente 🛡️
Params y Headers	Requiere manipulación manual de URL ⚠️	Se pasan fácilmente como objetos 🛠️

```
import axios from 'axios'
```

Preparando el entorno con pnpm

Usaremos pnpm por ser extremadamente rápido y eficiente.



```
# 1. Crear un proyecto Vue 3 en blanco
> pnpm create vue@latest consumo-api

# 2. Entrar a la carpeta generada
> cd consumo-api

# 3. Instalar dependencias base
> pnpm install

# 4. Instalar Axios (El cliente HTTP)
> pnpm add axios

# 5. Levantar el servidor de desarrollo
> pnpm dev
```

Las 3 variables esenciales

Todo consumo de API requiere controlar tres estados en la interfaz.



```
<script setup>
```

```
import { ref } from 'vue'
```

```
const users = ref([])
```

```
const loading = ref(false)
```

```
const error = ref(null)
```

```
</script>
```



Estado de Datos (users): Inicia como array vacío []. Aquí guardaremos el JSON que responda la API.



Estado de Carga (loading): Inicia en false. Nos dirá si la petición está en curso para mostrar un spinner o 'Cargando...'.



Estado de Fallo (error): Inicia en null. Si el servidor falla, guardaremos aquí el mensaje de error.

Ejecutando la petición HTTP

Usamos asincronismo para no congelar la pantalla.

```
async function fetchUsers() {  
  const response = await axios.get('https://reqres.in/api/users', {  
    params: { page: 1 },  
    headers: { 'x-api-key': 'reqres-free-v1' }  
  })  
  
  users.value = response.data.data  
}
```

La ejecución se pausa en esta línea hasta que el servidor responda, permitiendo que la UI siga interactiva.

¿Por qué dos veces 'data'?

1. El primer .data es el cuerpo de la respuesta que nos entrega Axios.
2. El segundo .data es cómo la API específica (Reqres) envuelve su lista de usuarios.

¿Qué nos devuelve Axios?

Axios envuelve el JSON del servidor en un objeto de control muy completo.

<code>data: { ... },</code>	El JSON real enviado por la API (Cuerpo)
<code>status: 200,</code>	Código HTTP (200 = OK, 404 = Not Found, 500 = Error)
<code>headers: { ... },</code>	Metadatos del servidor (ej. Tipo de contenido)
<code>config: { ... }</code>	Configuración original que enviamos en la petición

Personalizando la petición: Params y Headers

Params

- Viajan de forma visible en la URL (ej: ?page=1).
- Se utilizan para filtrar, ordenar o paginar resultados.

Headers

- No van en la URL, pero sí forman parte de la petición HTTP.
- Se utilizan para metadatos, configuraciones o enviar credenciales (API Keys).

```
const response = await axios.get('https://reqres.in/api/users', {  
  params: { page: 1 },  
  headers: { 'x-api-key': 'reqres-free-v1' }  
})
```

Resiliencia con try...catch

Las redes fallan. Así evitamos que nuestra aplicación colapse.

1

```
loading.value = true           // Iniciamos el estado de carga
error.value = null             // Limpiamos errores previos
```

2

```
try {
  const response = await axios.get('https://reqres.in/api/users', {
    params: { page: 1 },
    headers: { 'x-api-key': 'reqres-free-v1' }
  })
```

3

```
  users.value = response.data.data // ÉXITO: Guardamos los datos
} catch (err) {
  error.value = 'Falló la carga' // ERROR: Guardamos el mensaje
```

4

```
} finally {
  loading.value = false // FINAL: Apagamos el loading SIEMPRE
}
```

Entendiendo el error exacto

El objeto 'err' de Axios nos indica en qué punto falló la comunicación.

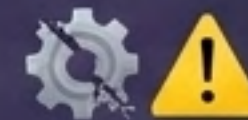
```
catch (err) {  
  if (err.response) {  
    error.value = `El servidor falló con código: ${err.response.status}`  
  
  } else if (err.request) {  
    error.value = 'No hubo respuesta del servidor (Error de red)'  
  
  } else {  
    error.value = 'Error al construir la petición'  
  }  
}
```



El servidor recibió la petición y respondió con un error (ej. 404, 500)



La petición se envió, pero no hay internet o el servidor está caído



Error interno de configuración antes de que saliera la petición

Conectando el estado a la vista

La UI reacciona automáticamente a las tres variables mediante directivas.

```
<template>
  <p v-if="loading">⌚ Cargando usuarios...</p>
  <p v-else-if="error" class="error">🔴 {{ error }}</p>
  <ul>
    <li v-for="user in users" :key="user.id">👤 {{ user.first_name }} - {{ user.email }}</li>
  </ul>
</template>
```

⌚ Cargando usuarios...

🔴 El servidor falló con código: 404

👤 George - george.bluth@reqres.in
👤 Janet - janet.weaver@reqres.in

La Foto Completa: ApiExample.vue

Todo integrado. Usamos **onMounted** para **ejecutar la función** apenas se abre la pantalla.

```
<script setup>
import { ref, onMounted } from 'vue'
import axios from 'axios'

const users = ref([]); const loading = ref(false); const error = ref(null)

async function fetchUsers() {
  loading.value = true; error.value = null
  try {
    const response = await axios.get('https://reqres.in/api/users', {
      params: { page: 1 },
      headers: { 'x-api-key': 'reqres-free-v1' }
    })
    users.value = response.data.data
  } catch (err) {
    error.value = err.response ? `Error: ${err.response.status}` : 'Error de red'
  } finally {
    loading.value = false
  }
}
```

onMounted(fetchUsers)
</script>



GATILLO: Se ejecuta al cargar el componente

¿Qué es JWT? (JSON Web Token)

Tu "pase de acceso" para consumir APIs protegidas.

Regla de Oro: ¡Un JWT NO es una contraseña!
No debe contener información sensible.

eyJhbGciOiJIUz... • eyJzdWIiOiIxMjM... • SfIKxwRJSMeK...

Header:

Tipo y algoritmo usado.

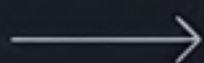
Payload:

Claims: datos mínimos
como ID, rol o permisos.

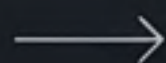
Signature:

Firma criptográfica del
servidor para evitar
alteraciones.

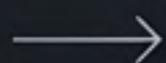
Login Exitoso



Servidor entrega JWT



Frontend conserva
temporalmente el JWT



Frontend lo envía en
futuras peticiones

Enviando el JWT con Axios

Usamos el Header 'Authorization' y el estándar 'Bearer'.

```
const token = 'eyJhbGciOiJIUzI1NiIs... ' // Token guardado en el cliente

const response = await axios.get('https://api.ejemplo.com/perfil', {
  headers: {
    Authorization: `Bearer ${token}`
  }
})
```



MDN Web Docs

[401 Unauthorized]

No enviaste el token, es inválido o expiró.



MDN Web docs

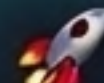
[403 Forbidden]

El token es válido (sabemos quién eres), pero NO tienes el rol/permisos necesarios para esta ruta.

Resumen de la Sesión

Has dominado el **ciclo de vida de los datos**.

- ✓ **Axios**: Cliente HTTP ágil para la comunicación web.
- ✓ **ref()**: Variables reactivas que controlan la interfaz (users, loading, error).
- ✓ **async/await**: Manejo de promesas sin bloquear la UI.
- ✓ **try...catch**: Manejo de fallos de red o respuestas erróneas del servidor.
- ✓ **JWT y Headers**: Acceso autorizado a rutas protegidas mediante **Bearer**.

 **En la próxima clase conectaremos Vue 3 con Firebase: Autenticación real, Base de Datos en la nube y sincronización en tiempo real.**